

게임 수학

동명대학교 게임공학과

강영민

무엇을 다룰 것인가

- 벡터와 행렬

- 벡터의 의미

- 벡터(vector)는 ‘나르다’라는 의미의 라틴어 동사 ‘vehere’에서 유래
 - ‘무엇인가를 나르는 것’이라는 의미
 - 벡터라는 것은 무엇인가를 옮겨 놓는 역할을 수행한다.

- 수학과 물리학에서의 개념

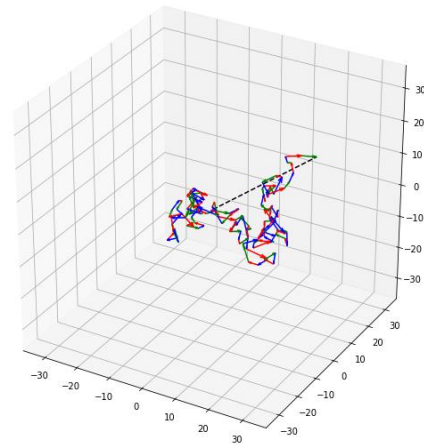
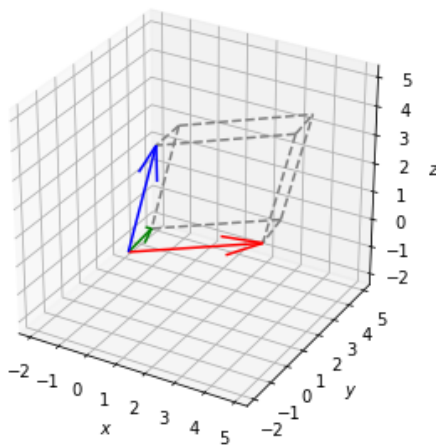
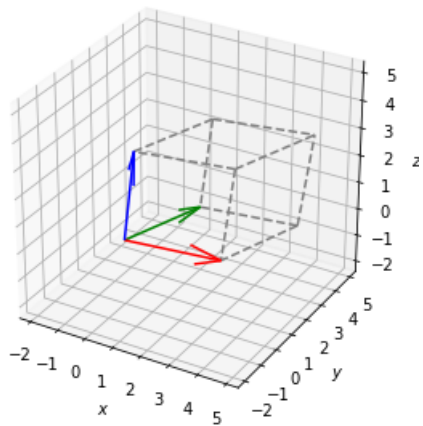
- 크기와 방향으로 결정되는 양(量, quantity)
 - 방향량(方向量)이라고도 함.
 - 예: 힘(force)은 크기만으로는 그 성질을 온전히 표현할 수 없고, 방향도 같이 고려해야 하므로 벡터로 표현된다.

왜?

- 게임은 벡터와 행렬로 데이터를 표현하고 조작한다

벡터

수학자들은 자연 현상의 많은 것들이 수로 표현될 수 있음을 알았는데, 하나의 숫자로 충분히 표현할 수 있지만 하나 이상의 수가 필요한 경우도 있다. 이러한 양을 벡터(vector)라고 부른다.



이 강의에서 무엇을 다룰까?

게임 세계를 움직이는 두 가지 수학적 도구를 배웁니다.



벡터 (Vector)

방향과 크기를 가진 수학적 객체. 캐릭터의 이동, 힘, 속도를 표현하는 기본 단위.

행렬 (Matrix)

2차원으로 배열된 수. 회전, 이동, 확대 등 공간 변환을 담당하는 강력한 도구.

왜 게임에 수학이 필요한가?

게임 속 모든 움직임, 충돌, 빛의 반사는 벡터와 행렬 연산의 결과입니다.

 캐릭터 이동

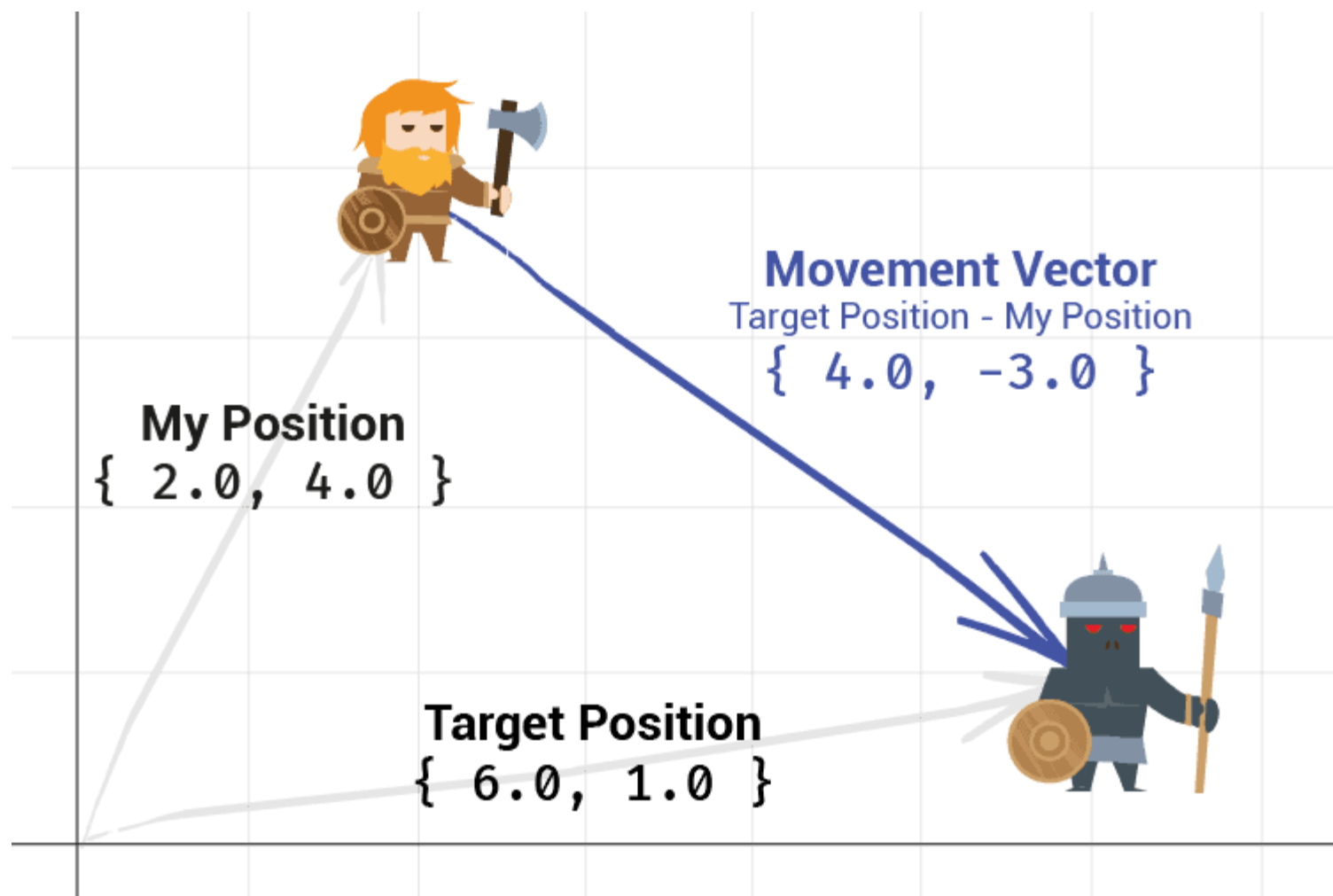
 충돌 감지

 3D 회전

 조명 계산

벡터

수학자들은 자연 현상의 많은 것들이 수로 표현될 수 있음을 알았는데, 하나의 숫자로 충분히 표현할 수 있지만 하나 이상의 수가 필요한 경우도 있다. 이러한 양을 벡터(vector)라고 부른다.



행렬은 왜?

- 행렬은 2차원으로 배열된 수
 - 벡터의 집합으로 해석 가능
 - 벡터와 곱해져서 새로운 벡터 만들 수 있음
 - 벡터를 바꾸어 놓는 연산자 - 트랜스포머?
 - 행렬은 벡터를 조작하는 도구

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{ij} & \cdots & a_{in} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mj} & \cdots & a_{mn} \end{bmatrix}$$

게임을 넘어서 — AI와 공학의 핵심 언어

선형대수는 게임 엔진뿐만 아니라, 인공지능·컴퓨터 비전·로보틱스 등 현대 공학 전반을 지탱하는 수학입니다.



딥러닝 / 인공지능

신경망의 가중치는 거대한 행렬. 학습은 행렬 연산의 반복이다.



컴퓨터 비전

이미지를 픽셀 행렬로 변환하고, 특징을 벡터로 추출해 사물을 인식한다.



로보틱스 / 시뮬레이션

로봇 관절의 각도와 힘을 행렬로 계산해 정밀한 동작을 구현한다.



데이터 과학

수백만 개의 데이터를 벡터 공간에 표현하고 패턴을 찾아낸다.

행렬은 왜 중요한가?

행렬은 2차원으로 배열된 수로, 벡터를 변환하는 강력한 연산자입니다.



벡터의 집합

여러 벡터를 하나의 행렬로 묶어 표현 가능

공간 변환 도구

회전·확대·이동 등 모든 3D 변환은 행렬 곱 하나로 표현

트랜스포머!

AI의 핵심 아키텍처 "Transformer"도 본질적으로 행렬 연산의 집합체

실습은 어떻게 진행하나

- 파이썬 프로그래밍
 - 수학적 개념을 실제로 코드로 구현해 봄
 - 벡터의 연산, 행렬의 연산 결과를 숫자와 시각 정보로 확인
- 파이썬 코딩을 하는 방법
 - 다양한 방법이 존재
 - Python interpreter 설치 + 내게 맞는 IDE (예 - visual studio code)
 - 우리가 사용할 방법
 - Google Colab 사용

실습 환경 — 파이썬과 구글 코랩

수학적 개념을 직접 코드로 구현하고, 결과를 숫자와 시각 정보로 즉시 확인합니다.



설치 불필요

웹 브라우저만 있으면 바로 파이썬 실행 가능



쉬운 협업

파일 공유와 공동 작업이 구글 드라이브처럼 간단



고성능 계산

GPU·TPU를 무료로 사용해 대용량 연산도 거뜰히



라이브러리 선택재

NumPy, Pandas, TensorFlow 등 핵심 패키지 기본 제공



바로 시작하기: colab.research.google.com

구글 코랩 사용법

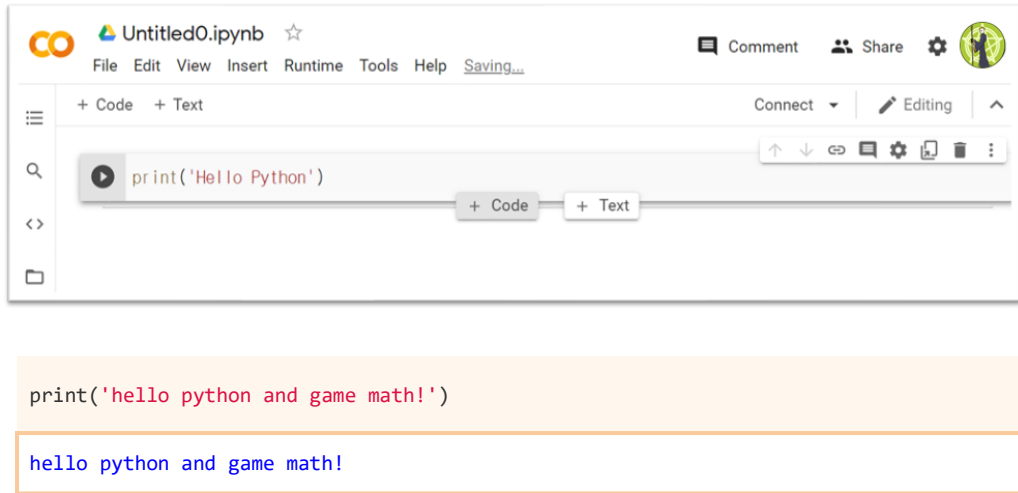
- 장점

- 파이썬을 설치하지 않고도 웹 환경의 주피터 노트북에서 파이썬 사용 가능
- 다른 사용자와의 파일 공유가 가능하며 협업을 통한 개발도 손쉽게 할 수 있다.
- 넘파이, 판다스, 사이킷런, 텐서플로우 등의 패키지가 미리 설치된 환경 제공
- 클라우드에서 제공하는 GPU와 TPU를 사용하여 고성능 계산을 할 수 있다.

- 사용방법

- 구글 계정이 있다면 아래 사이트에 접속해서 사용
- <https://colab.research.google.com>

사용 예시



The screenshot shows a Jupyter Notebook titled 'Untitled0.ipynb'. The code cell contains the following Python code:

```
print('Hello Python')
```

The output of the code cell is:

```
hello python and game math!
```



시스템 관리 작업에 신경 쓸 필요 없이 관심 있는 이론과, 이를 구현하는 코드에만 집중할 수 있을 것이다.

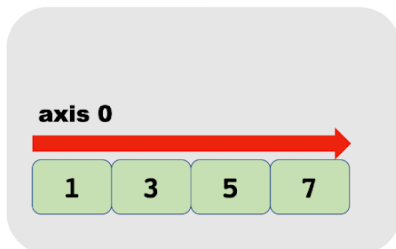
사용할 패키지 - 넘파이

- 넘파이는 파이썬에서 수치 데이터를 다루는 패키지
 - 가장 기본적이고 강력한 패키지
 - 데이터 분석이나 기계 학습에서 활발히 활용
 - 데이터 분석을 위한 패키지인 판다스Pandas
 - 머신러닝을 위한 Scikit-learn, Tensorflow 등이 넘파이 위에서 작동

```
import numpy as np
my_array = np.array( [ 1, 2, 3 ] )
```

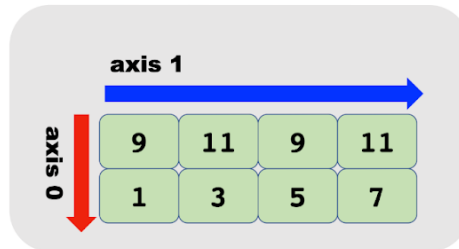
numpy의 별칭으로 np를 지정함
np는 numpy의 별칭

1D 배열



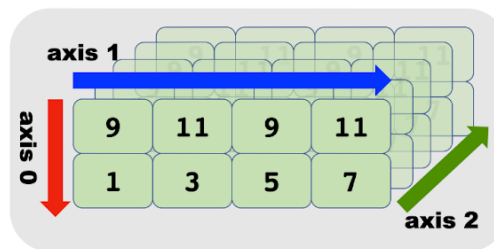
shape = (4,)

2D 배열



shape = (2,4)

3D 배열



shape = (2,4,5)

넘파이 배열과 파이썬 리스트의 차이

- 리스트는 벡터 데이터가 아님

```
store_a = [20, 10, 30] # 매장 A의 매출  
store_b = [70, 90, 70] # 매장 B의 매출  
list_sum = store_a + store_b  
list_sum
```

```
[20, 10, 30, 70, 90, 70]
```

- 넘파이의 배열은 벡터, 행렬을 표현하는 데에 적합

```
import numpy as np # 넘파이를 임포트하고 np라는 별칭 지정  
np_store_a = np.array(store_a) # store_a 리스트를 넘파이 배열로 변환  
np_store_b = np.array(store_b) # store_b 리스트를 넘파이 배열로 변환  
array_sum = np_store_a + np_store_b  
array_sum
```

```
array([ 90, 100, 100])
```

넘파이 배열의 다양한 속성

- 아래 넘파이 배열 속성을 점검하여 계산이나 오류 파악에 도움을 얻을 수 있다

속성	설명
ndim	배열 축 혹은 차원의 수
shape	배열의 차원으로 (m, n) 형식의 튜플. 이 때, m과 n은 각 차원별 원소의 크기
size	배열 원소의 수
dtype	배열에 담긴 원소의 자료형
itemsize	배열에 담긴 원소의 크기를 바이트 단위로 나타냄
data	배열의 실제 원소들을 가지고 있는 메모리 버퍼
stride	배열 각 차원별로 다음 원소로 점프하는 데에 필요한 거리를 바이트로 표시

넘파이의 성공 요인

- 배열 프로그래밍 – array programming



배열 프로그래밍의 성능 확인 1/2

```
import numpy as np
import time
arr_a = np.random.rand(100)
arr_b = np.random.rand(100)
result = np.zeros(100)
start = time.time()
for i in range(len(values)):
    result[i] = arr_a[i] * arr_b[i]
end = time.time()
print('소요시간: ', end - start)
```

명시적인 작업 지시 : C 언어 스타일

소요시간: 0.0012476444244384766

배열 프로그래밍의 성능 확인 2/2

```
start = time.time()
result = arr_a * arr_b          # 벡터화 연산 *
end = time.time()
print('소요시간: ', end - start)
```

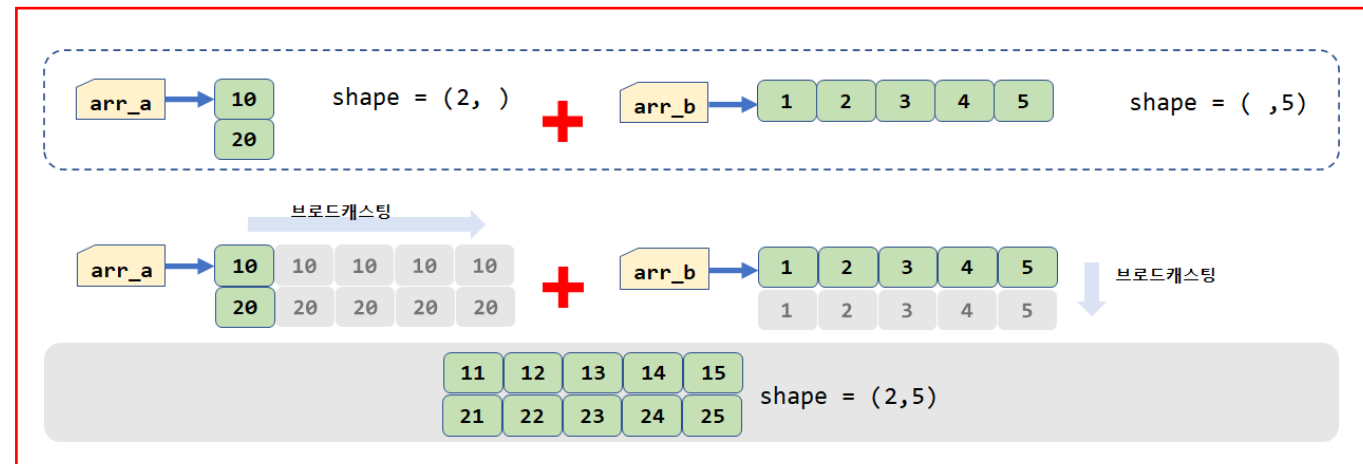
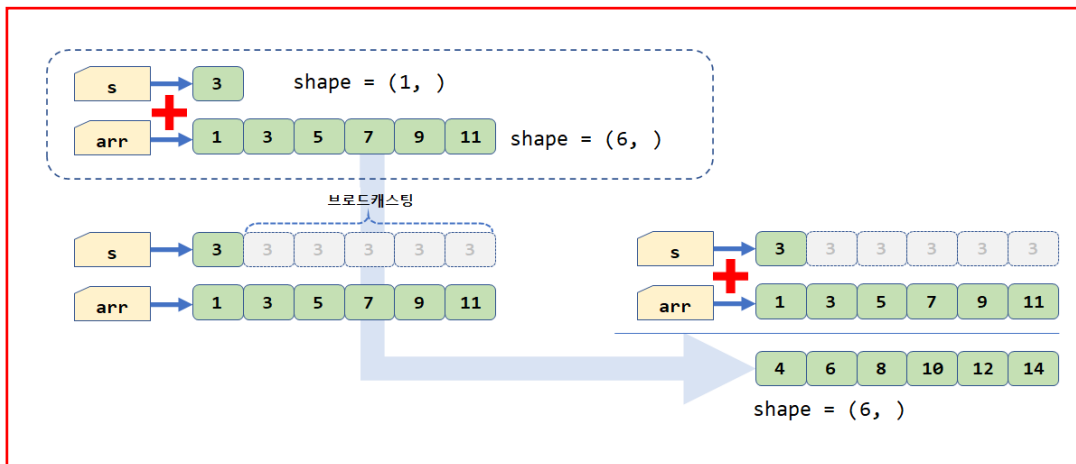
소요시간: 0.0000588893890380859

데이터 개수에 따른 성능 차이

N	10K	20K	40K	100K	200K	400K	1M	2M	3M	4M	5M
성능	46	148	126	143	150	179	187	195	177	182	206

넘파이 장점 - 브로드캐스팅

벡터화 연산은 두 피연산자 동일한 shape을 갖고 있을 때 대응되는 원소들에 대해 각각 연산이 적용된다. 그런데, **브로드캐스팅**은 이러한 벡터화 연산이 다른 shape을 가지고 있을 때에도 연산이 가능하게 만들어 준다.



브로드캐스팅에 의한 성능 향상

```
import numpy as np
import time

n = 1000
arr_a = np.random.rand(n, 1)
arr_b = np.random.rand(1, n)
print(arr_a.shape, arr_b.shape)
```

```
# 브로드캐스팅과 벡터화 없이 더하기
start = time.time()
res_naive = np.empty((n, n))
arr_a_ex = np.empty((n, n))
arr_b_ex = np.empty((n, n))
for i in range(0, n):
    for j in range(0, n):
        arr_a_ex[j, i] = arr_a[j, 0]
        arr_b_ex[i, j] = arr_b[0, j]

for i in range(n):
    for j in range(n):
        # 두 배열의 원소별 합
        res_naive[i, j] = arr_a_ex[i, j] + arr_b_ex[i, j]
end = time.time()
print(n, 'x', n, ": ", end-start)
```

```
1000 x 1000 : 1.60617995262146
```

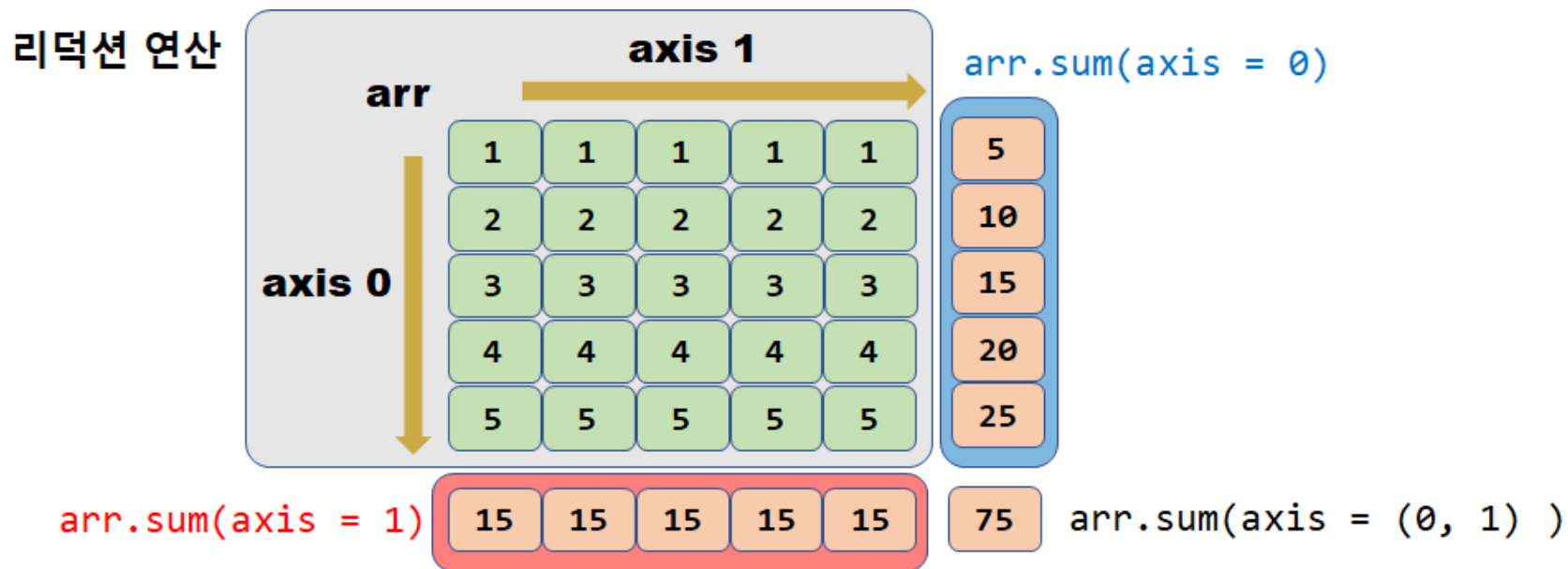


```
# 브로드캐스팅과 벡터화 연산으로 더하기
start = time.time()
res = arr_a + arr_b
end = time.time()
print(n, 'x', n, ": ", end-start)
```

```
1000 x 1000 : 0.0021255016326904297
```

넘파이 장점 - 리덕션

리덕션이라는 것은 여러 개의 수로 이루어진 배열을 하나의 수로 집계^{aggregation}하는 것이다. 넘파이는 이 집계를 위한 함수로 `sum()`, `mean()`, `min()`, `max()` 같은 함수를 제공



리덕션에 의한 성능향상

```
▶ import numpy as np
import time

n = 1000
arr = np.random.rand(n, n)
```

```
▶ sum_naive = 0
start = time.time()
for i in range(n):
    for j in range(n):
        sum_naive += arr[i,j]
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_naive, "계산 시간: ", end-start)
```

```
→ 1000 x 1000 배열의 원소 합 500048.24990126066
계산 시간: 0.39148378372192383
```



```
▶ start = time.time()
sum_reduction = np.sum(arr, axis = (0,1))
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_reduction, "계산 시간: ", end-start)
```

```
→ 1000 x 1000 배열의 원소 합 500048.2499012628
계산 시간: 0.00128173828125
```



시작하며

수학은 게임과 AI를 만드는 언어입니다

게임

이동·충돌·렌더링 모두 벡터와 행렬

AI / 딥러닝

신경망 학습의 본질은 행렬 연산

공학 전반

로보틱스·시뮬레이션·데이터 과학

파이썬 실습

NumPy로 직접 구현하며 체득



이 강의를 마치면 게임 엔진이 어떻게 3D 세계를 계산하는지, 그리고 AI가 어떻게 패턴을 학습하는지 수학적으로 이해하게 됩니다. 함께 시작해 봅시다! 🚀

애니메이션 — 뼈대 애니메이션(Skeletal Animation)의 수학

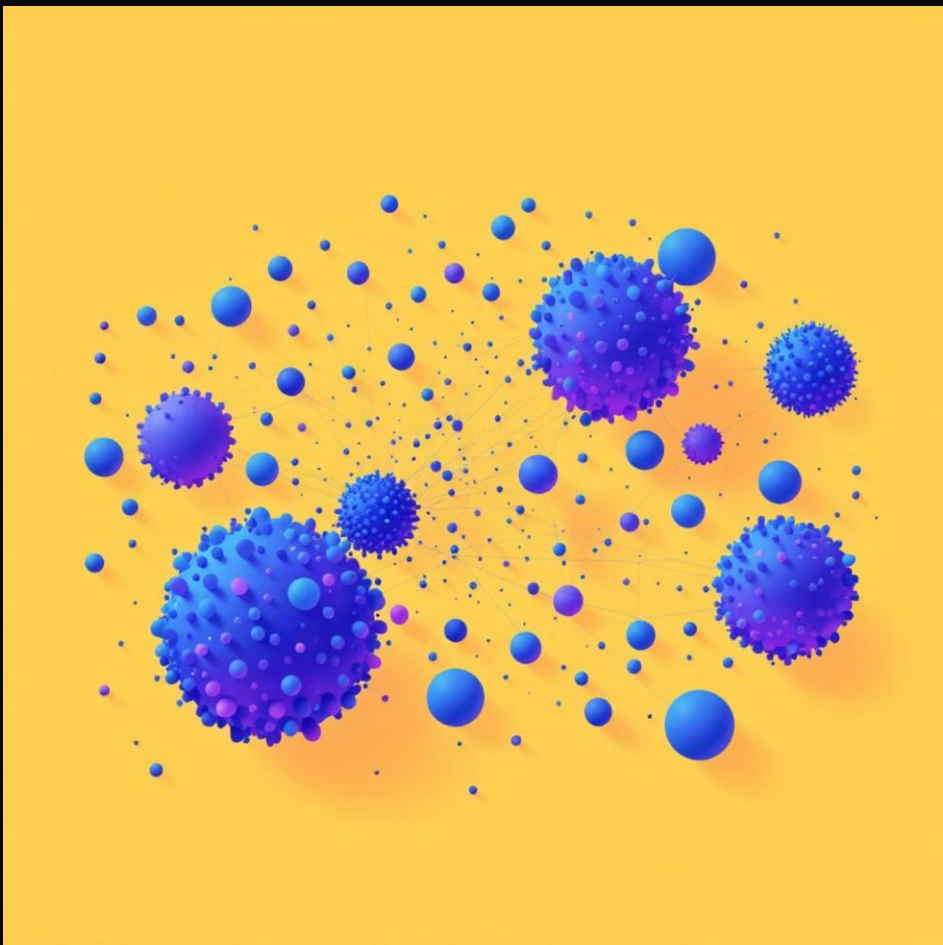
게임과 영화 속 생생한 캐릭터 움직임은 뼈대(스켈레톤)라는 가상 골격 시스템을 기반으로 합니다. 각 뼈의 회전 행렬과 위치 벡터를 계산하여 캐릭터의 자세와 움직임을 정의하죠.

두 애니메이션 프레임 사이의 자세를 자연스럽게 이어주는 보간(Interpolation) 기술과, 여러 애니메이션을 동시에 부드럽게 결합하는 블렌딩(Blending)을 통해 더욱 사실적인 움직임을 만들어냅니다.



데이터 과학 — 고차원 공간에서의 패턴 발견

수백만 개의 데이터 포인트를 벡터로 표현하고, 복잡한 패턴을 추출하는 수학적 기법을 알아봅니다.



01

데이터 벡터화

텍스트, 이미지, 사용자 행동 등 모든 데이터를 숫자의 배열(벡터)로 변환합니다.

02

차원 축소 (PCA)

수많은 특징(차원) 중 가장 중요한 요소를 찾아 데이터의 복잡성을 줄이고 시각화합니다.

03

클러스터링

유사한 특성을 가진 데이터 포인트들을 그룹(클러스터)으로 묶어 숨겨진 구조를 발견합니다.

04

유사도 계산

두 벡터 간의 거리나 각도를 측정하여 데이터가 얼마나 서로 닮았는지 정량적으로 파악합니다.

로보틱스 — 팔의 움직임을 계산하다

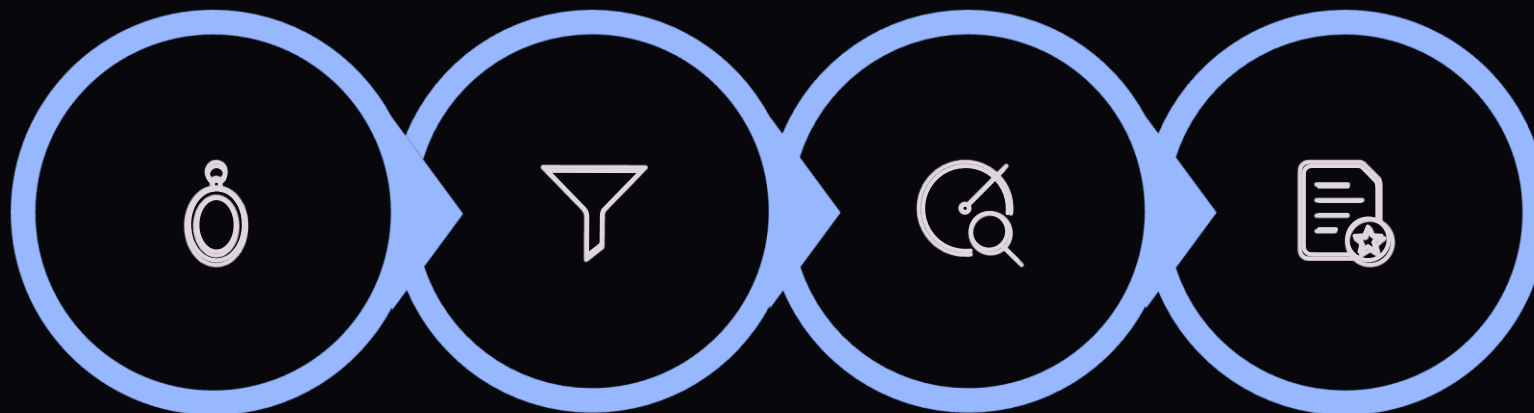
로봇 팔은 여러 개의 관절과 링크로 연결된 복잡한 시스템입니다. 각 관절의 움직임을 벡터로 표현하고, 이들이 어떻게 조합되어 로봇 팔의 최종 위치와 방향을 결정하는지 수학적으로 이해하는 것이 중요합니다.



이러한 원리는 정방향 운동학(Forward Kinematics)과 역방향 운동학(Inverse Kinematics)의 핵심이며, 모두 행렬 변환의 연쇄를 통해 로봇 팔의 복잡한 움직임을 제어합니다.

컴퓨터 비전 — 이미지 처리와 특징 추출

컴퓨터 비전은 이미지를 숫자 행렬로 변환하고, 행렬 연산을 통해 정보를 추출하는 학문입니다. 이는 게임의 시각 처리와 AI의 핵심 기반이 됩니다.



Original

Filtered

Edges

Feature Map

원본 이미지는 거대한 픽셀 행렬로 표현되며, 여기에 **필터(Filter)**라는 또 다른 작은 행렬을 곱하면 이미지를 흐리게 하거나 선명하게 하는 등의 다양한 변환이 가능합니다. 이 과정을 통해 중요한 **특징(Feature)**을 추출하고, 엣지를 감지하여 사물을 인식하는 기초를 마련합니다. 이는 딥러닝의 핵심 기술인 CNN(합성곱 신경망)의 기본 원리가 됩니다.

신경망 — 행렬 연산의 연속

딥러닝 신경망은 본질적으로 거대한 행렬 곱셈의 연쇄이며, 이 과정을 통해 학습하고 예측합니다.



이 다이어그램은 신경망의 **순전파(Forward Propagation)** 과정을 보여줍니다. 입력 데이터는 각 층의 가중치 행렬과 곱해지고 활성화 함수를 통과하며 다음 층으로 전달됩니다.

신경망 학습의 핵심인 **역전파(Backpropagation)**는 출력 오차를 바탕으로 가중치 행렬을 조정하는 과정으로, 역시 복잡한 행렬 연산과 미분(경사하강법)을 통해 이루어집니다.

물리 엔진 — 충돌 감지와 반응

게임 세계의 역동적인 상호작용은 벡터와 행렬을 통해 정확하게 계산됩니다.



AABB (Axis-Aligned Bounding Box)

가장 간단한 충돌 판정. 직사각형/직육면체 경계 상자로 물체를 감싸 계산을 단순화하여 성능을 최적화합니다.



구 충돌 (Sphere Collision)

물체를 구로 근사화하여 두 구 사이의 거리로 충돌을 판정합니다. 계산이 빠르고 물리 시뮬레이션에 유용합니다.

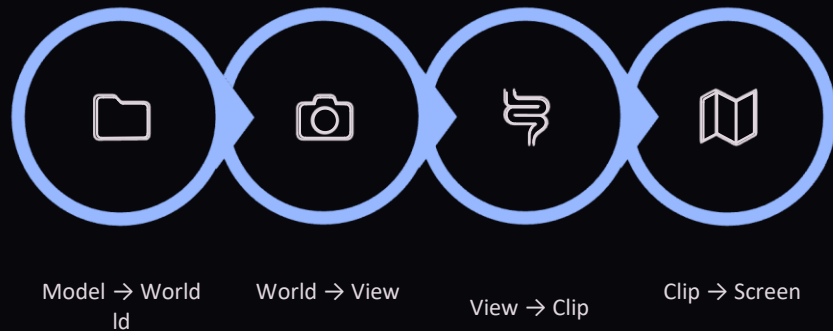


메시 충돌 (Mesh Collision)

물체의 실제 3D 모델(메시)을 사용하여 가장 정밀한 충돌을 감지합니다. 복잡한 형태의 물체에 적합합니다.

컴퓨터 그래픽스 — 3D 렌더링의 수학

3D 세계가 화면에 그려지는 복잡한 과정은 벡터와 행렬 없이는 불가능합니다.



3D 모델이 카메라를 통해 2D 화면으로 변환되는 과정을 시각적으로 보여줍니다. 각 단계는 행렬 변환을 통해 이루어집니다.

01

모델 좌표계 (Model Space)

객체 자체의 고유한 공간. 모든 3D 모델은 이 공간에서 정의됩니다.

02

월드 좌표계 (World Space)

여러 객체가 배치되는 가상의 전역 공간. 모델 행렬(Model Matrix)로 모델 좌표를 월드 좌표로 변환합니다.

03

뷰 좌표계 (View Space)

카메라의 시점에서 바라본 공간. 뷰 행렬(View Matrix)로 월드 좌표를 카메라 시점으로 변환합니다.

04

클립 좌표계 (Clip Space)

렌더링될 영역을 결정하는 공간. 투영 행렬(Projection Matrix)로 뷰 공간을 화면에 보이는 형태로 변환합니다.

실제 게임 엔진 사례 — Unity와 Unreal Engine

Unity와 Unreal Engine은 선형대수의 원리를 사용하여 복잡한 3D 게임 세계를 구축합니다.



게임 엔진의 핵심, Transform
게임 오브젝트의 모든 위치, 회전, 크기는 Transform 컴포넌트에 저장되며, 이는 내부적으로 벡터와 행렬로 관리됩니다.

```
import numpy as np

# 위치 (벡터)
position = np.array([10.0, 5.0, 0.0])

# 크기 (벡터)
scale = np.array([1.0, 1.0, 1.0])

# 회전 (쿼터니언 또는 회전 행렬)
# 예: Y축으로 90도 회전하는 쿼터니언
rotation_q = np.array([0.707, 0.0, 0.707, 0.0])
```

카메라가 세상을 보는 방법

뷰 행렬 (View Matrix) 3D 공간의 모든 오브젝트를 카메라의 시점에서 바라본 좌표계로 변환합니다. 플레이어의 시야에 맞춰 세상을 재정렬하는 역할을 합니다.	$V = T^{-1}R^{-1}$
프로젝션 행렬 (Projection Matrix) 3D 공간의 오브젝트들을 2D 화면에 투영하기 위해 변환합니다. 원근감을 적용하여 가까운 것은 크게, 먼 것은 작게 보이게 합니다.	$P = \text{Perspective or Orthographic Projection}$
쿼터니언 (Quaternion) 3D 회전을 표현하는 수학적 방법으로, 짐벌 락(Gimbal Lock) 문제 없이 부드럽고 안정적인 회전을 구현하는 데 사용됩니다.	

벡터의 표현

동명대학교 게임공학과

강영민

벡터의 개념

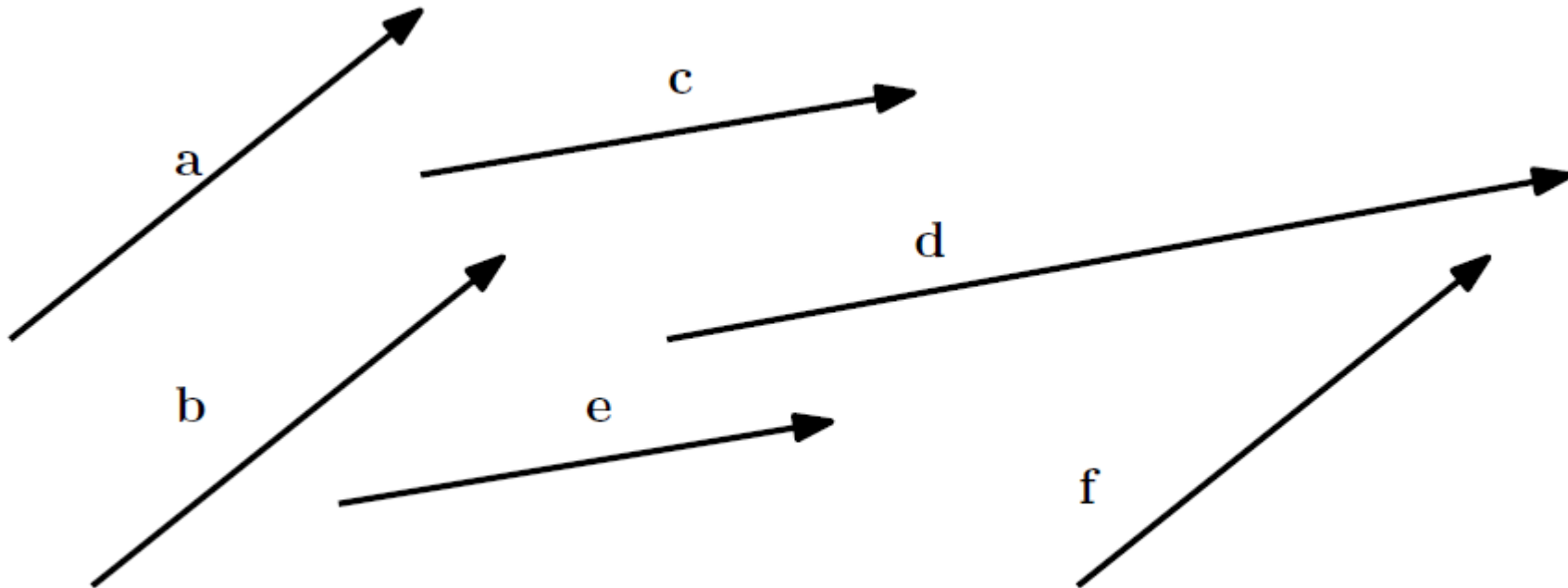
- 물리적 현상 등을 표현할 때 그 대상을 양^{quantity}로 표현
- 이 양은 스칼라 혹은 벡터
 - 스칼라: 오로지 크기만으로 완전히 그 양을 표현할 수 있음
 - 질량, 소요시간, 길이, 열량
 - 대소 비교가 가능
 - 벡터: 크기와 함께 방향도 중요한 양
 - 하나의 숫자로 표현할 수 없음
 - 힘, 속도, 변위
- 속도^{velocity}와 속력^{speed}
 - 속도: 벡터
 - 속력: 스칼라

이과가 싫어합니다



동등벡터 찾기

- 다음 중 동등벡터들을 찾아 묶어 보자

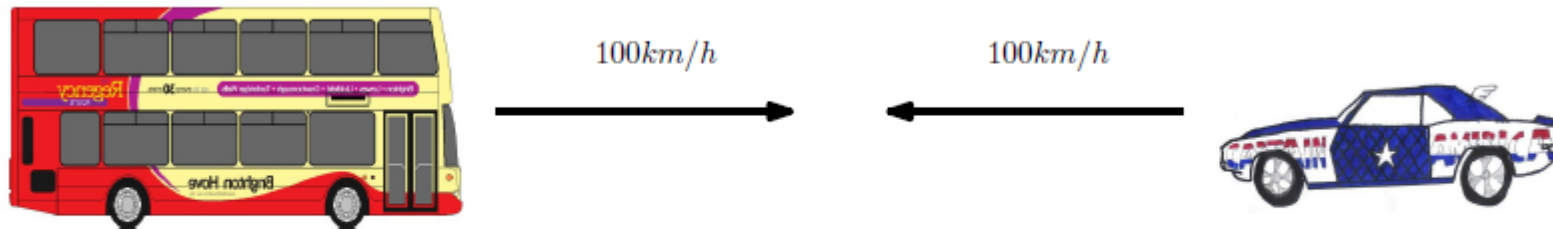


동등벡터 찾기

벡터의 표현

- 속도와 속력

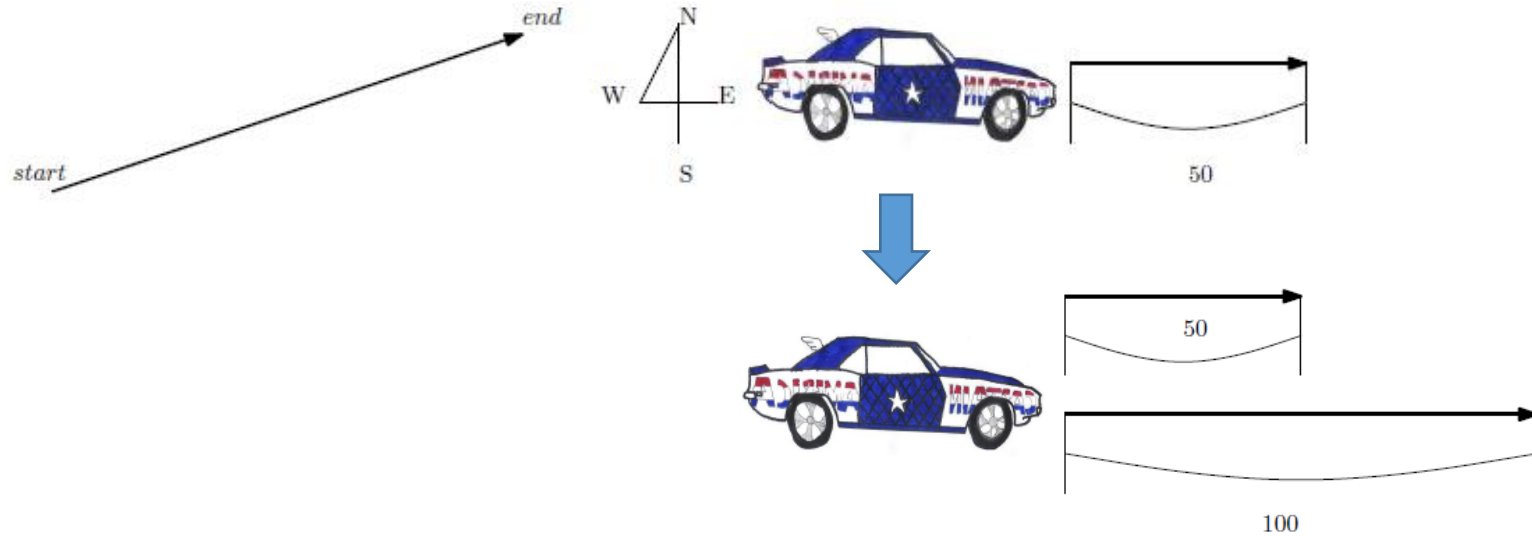
- 속력은 숫자로 표현하면 그만이지만, 속도는 숫자 하나로 표현 불가



동일한 속력으로 서로 마주 보며 달리는 차량의 속도

화살표를 이용한 벡터 표현

- 벡터를 표현하는 가장 직관적인 방법
 - 화살표: 시작하는 점과 끝나는 점으로 구성
 - 화살표의 방향은 벡터가 작용하는 방향
 - 화살표의 길이는 벡터가 가진 크기



속력이 두 배로 늘어난 자동차의 속도

동등벡터 equivalent vector

- 벡터의 표기법 $\vec{a}$, a

- 동등벡터

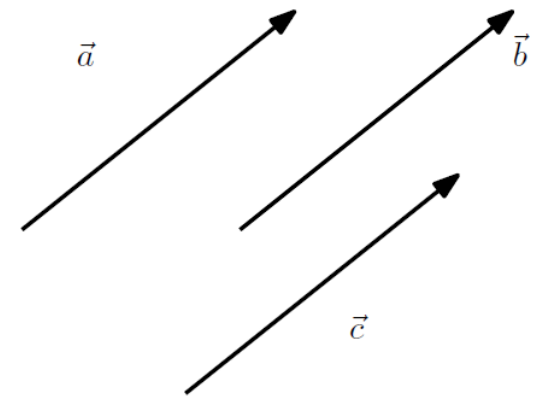
- 크기와 방향이 같은 벡터들은 모두 동등한 벡터로 간주

- 동등하다: 같은 양을 의미

- 예) 물 5kg과 쌀 5kg은 같은 대상을 가리키는 것은 아니지만, “무게 ” 라는 양의 측면에서 동등

- 두 벡터가 동등하다는 것은 두 벡터가 같은 존재라는 것이 아님

- 벡터로 표현되는 양이 같다는 의미



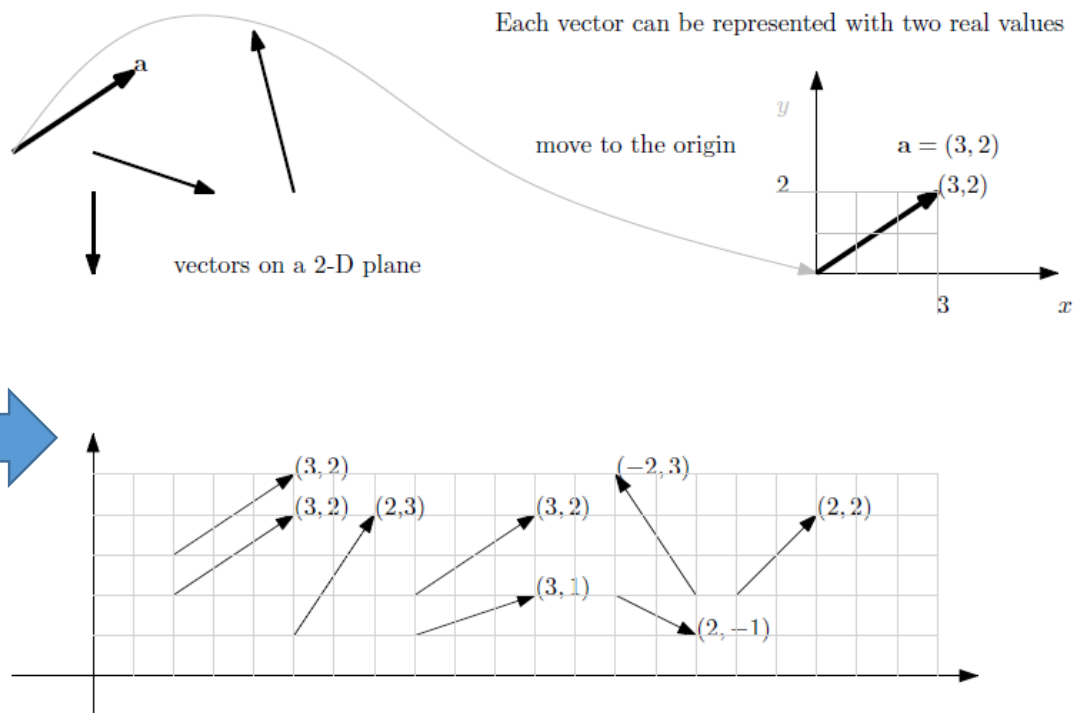
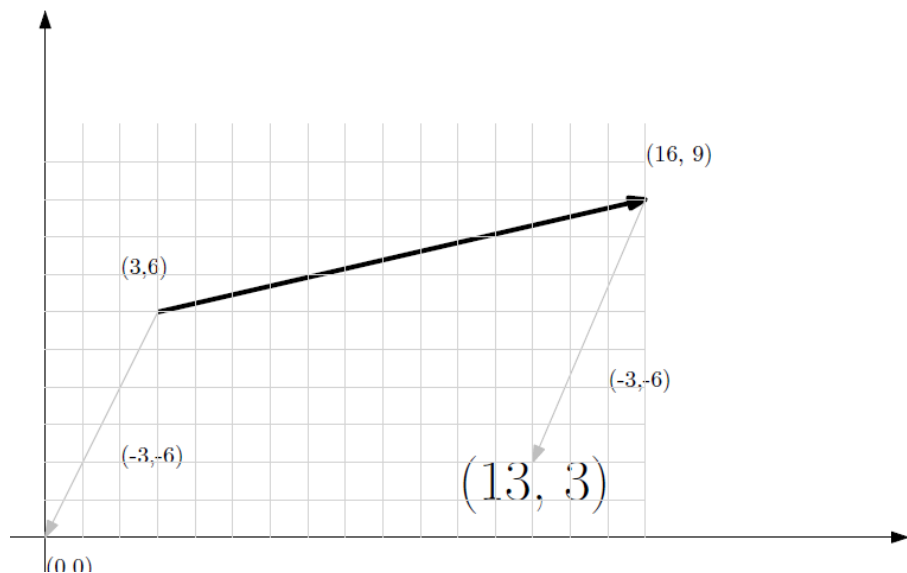
동등벡터

벡터의 수학적 표현

- 벡터를 조금 더 형식을 갖춰 표현하는 방법
 - 수학적으로 연산이 편리하도록 표현
 - 벡터의 화살표 표현
 - 화살표가 그려지는 공간의 차원에 따라 좌표의 원소 개수 결정
 - 벡터의 차원 = 공간의 차원
 - 벡터가 n 차원이라면
 - 이 벡터는 n 개의 숫자로 표현되는 좌표 형태로 다룰 수 있을 것
 - n -튜플^{tuple}
- $$\mathbf{v} = (v_1, v_2, v_3, \dots, v_n)$$
- n 개의 차원을 가진 공간에서 그려지는 화살표 = n 차원 벡터 = n -튜플

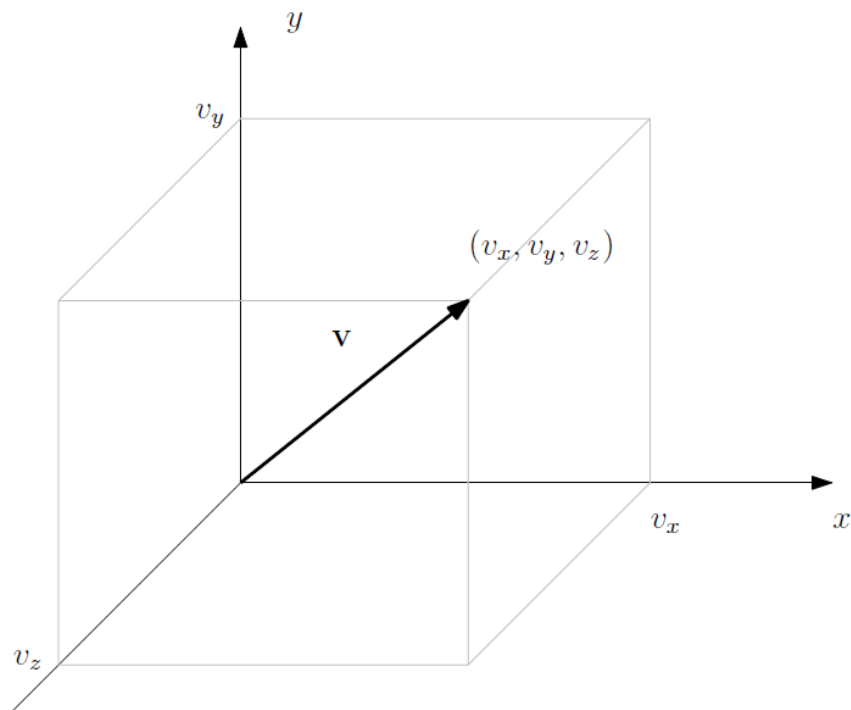
2차원 벡터의 예

- 시작점 (x_s, y_s), 끝점 (x_e, y_e)
 - 네 개의 숫자로 표현할 수 있음
 - 동등 벡터는 모두 같은 형태로 표현되어야 함
 - 이를 위해서는 시작점을 원점으로 옮기면 동등 벡터는 모두 같은 끝점을 가짐
 - 이 끝점이 이들 동등벡터를 대표하는 양

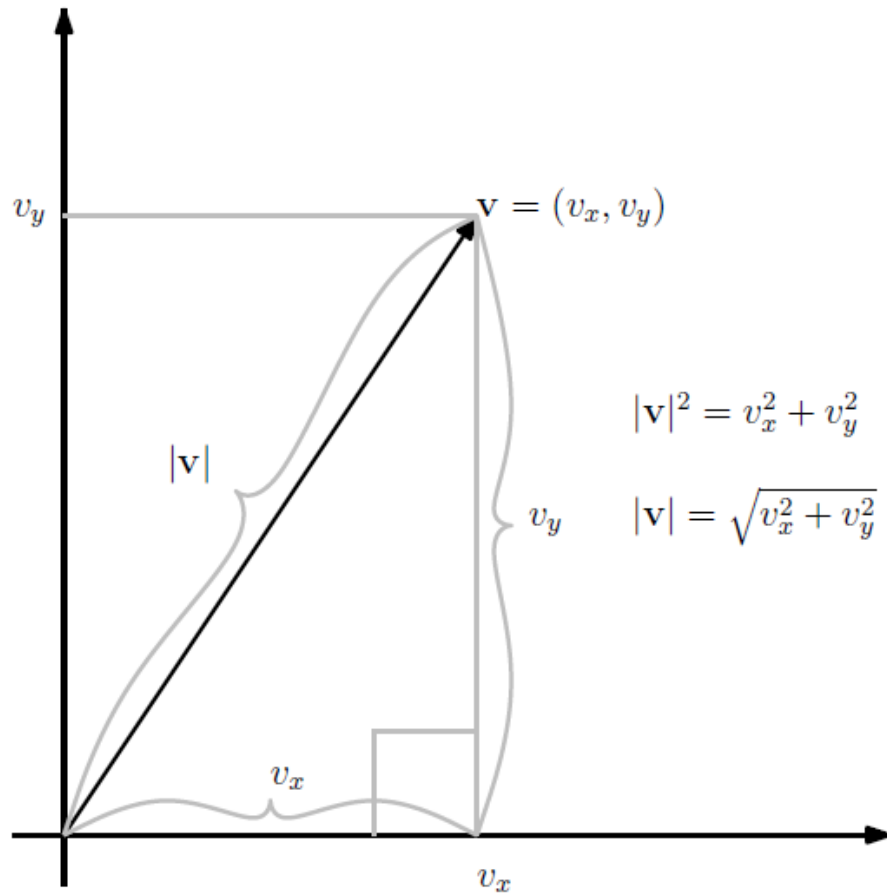


3차원 벡터

- 3차원 벡터는 2차원 벡터에 축을 하나 더하기만 하면 된다



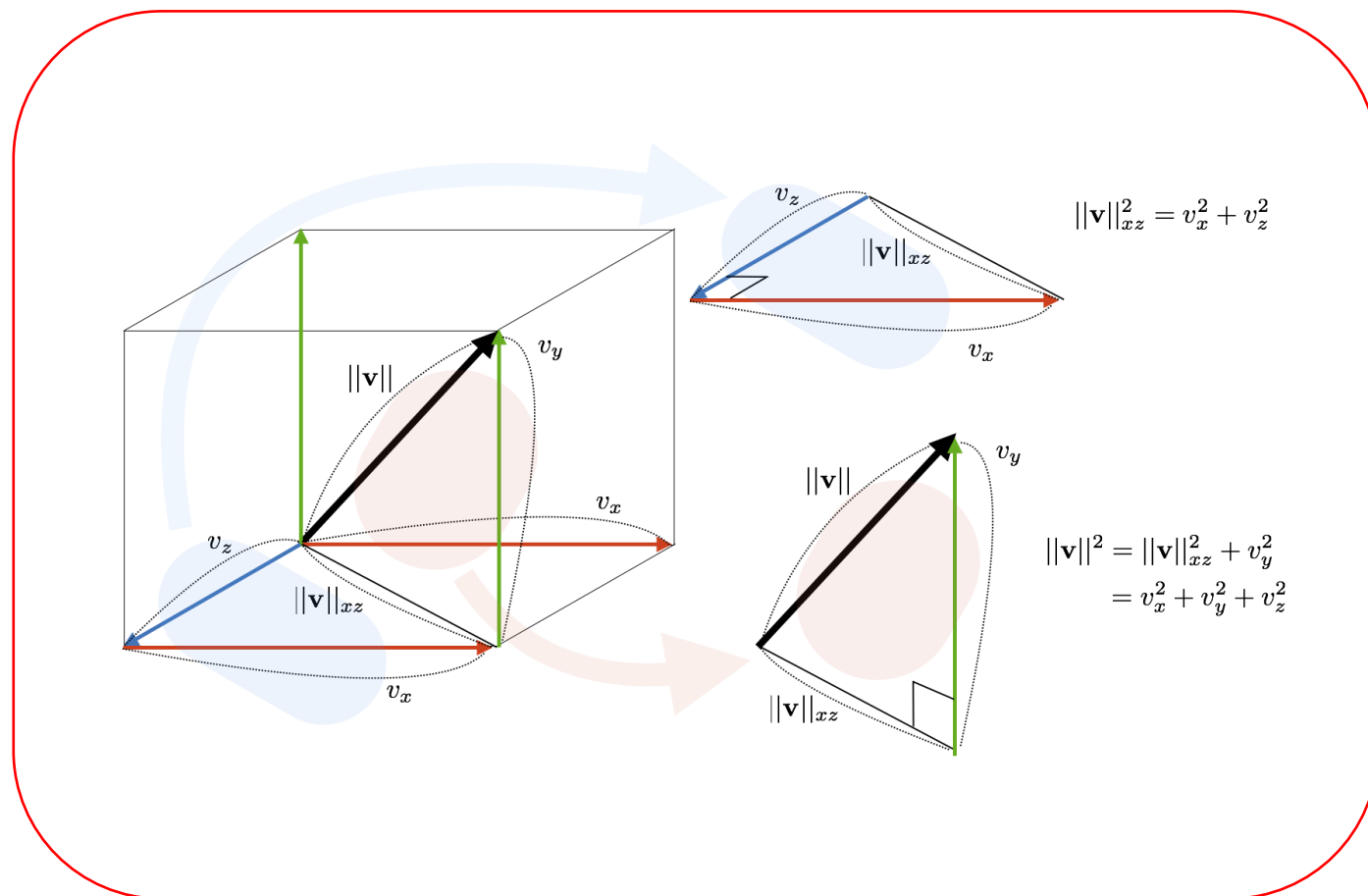
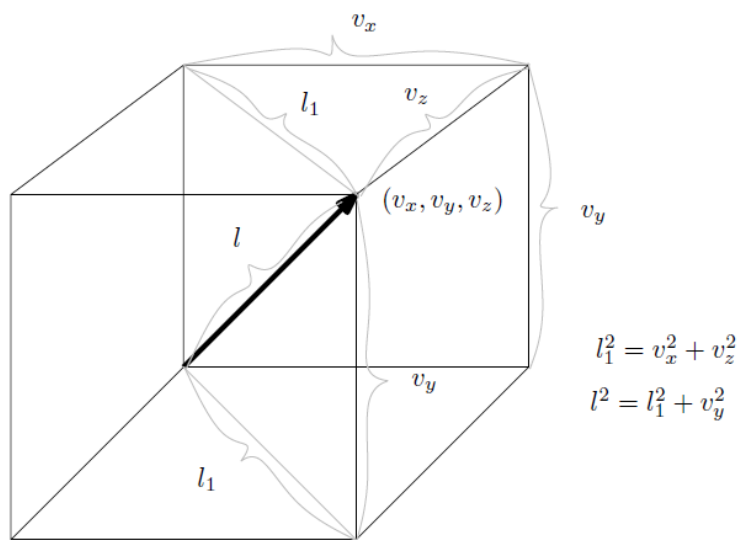
2차원 벡터의 크기



- 왼쪽에 보이는 벡터 $\mathbf{v}$ 의 크기는?
 - 좌표값은 각 축으로의 길이
 - 2차원의 예에서
 - X좌표는 밑변
 - Y좌표는 높이
 - 벡터의 크기는 이러한 밑변과 높이를 가지는 직각삼각형의 빗변 길이
 - 피타고라스 정리로 간단히 계산
- 크기는 언제나 양의 값
- 이 값을 노름(norm, 놈)이라 함

3차원 벡터의 크기

$$\mathbf{v} = (v_x, v_y, v_z)$$
$$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$



벡터 노름 - 일반화

- 피타고라스 정리를 이용하여 구한 벡터의 노름
 - 특별한 예: 2차 노름, l_2 노름
- 노름의 일반화

$$||\mathbf{x}||_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

- p 값에 따라 l_p -노름이라 부름
- p=1
 - 맨해튼 거리라고도 함
- p가 무한대
 - 원소중에 제일 큰 값을 얻음 (max)

벡터 노름 계산해 보기

```
▶ import numpy as np          # numpy의 별명을 np로 함  
  
v = np.array([1, 2, 3])      # 세 개의 원소를 가진 배열로 3차원 벡터 표현  
print(v.ndim, v.shape)      # 넘파이 배열의 ndim과 shape 출력
```

```
➡ 1 (3,)
```

```
▶ def norm(v, p=2) :  
    dim = v.shape[0]          # 원소의 개수를 담고 있음 = 벡터의 차원  
    acc = 0  
    for i in range(dim):      # 벡터의 차원만큼 모든 원소의 p 제곱 구해 누적  
        acc += v[i]**p  
    return acc**(1/p)         # 전체에 1/p 제곱을 적용하여 p-노름 계산
```

```
▶ v_norm_1 = norm(v, p=1)     # 맨해튼 노름 (L1)  
v_norm_2 = norm(v, p=2)     # 유클리드 노름 (L2)  
v_norm_3 = norm(v, p=3)     # 3-노름  
v_norm_4 = norm(v, p=4)     # 4-노름  
print(v_norm_1, v_norm_2, v_norm_3, v_norm_4)
```

```
➡ 6.0 3.7416573867739413 3.3019272488946263 3.1463462836457885
```

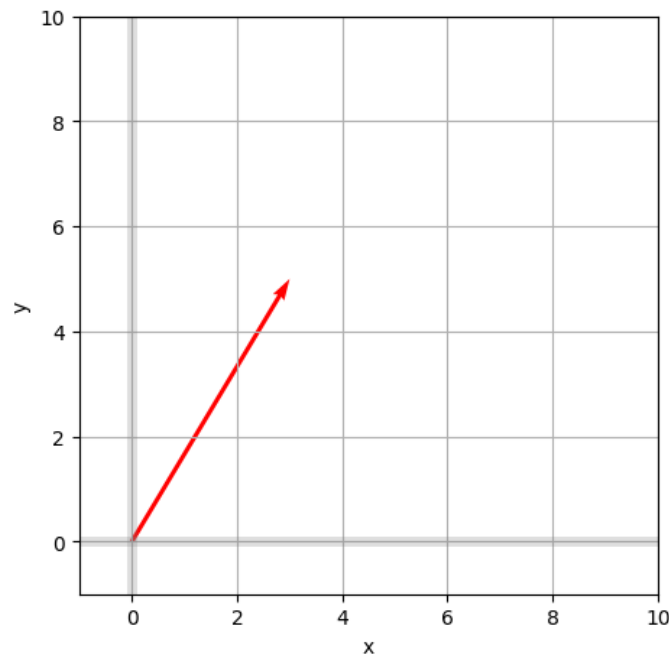
$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

가시화 도구

<https://github.com/dknife/linalg/blob/main/tool/visualizer.py>

```
!wget https://raw.githubusercontent.com/dknife/linalg/main/tool/visualizer.py  
from visualizer import *  
!rm visualizer.py
```

```
# (3,5) 벡터를 가시화 하자  
  
my_axis = axis2d(x=[-1,10], y=[-1,10])  
my_vector = np.array([3,5])  
draw_vec2d(my_axis, my_vector)
```



행렬의 기하학적 변환 — 회전, 확대, 이동

행렬은 게임 그래픽에서 3D 오브젝트를 화면에 배치하고 움직이는 핵심 도구입니다. 벡터를 회전시키고, 크기를 조절하며, 위치를 이동 동시키는 과정을 시각적으로 살펴봅니다.



회전 (Rotation)

객체의 방향을 변경합니다. 특정 축을 중심으로 각도만큼 돌립니다.

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$



확대/축소 (Scaling)

객체의 크기를 변경합니다. 각 축 방향으로 크기 비율을 조절합니다.

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$



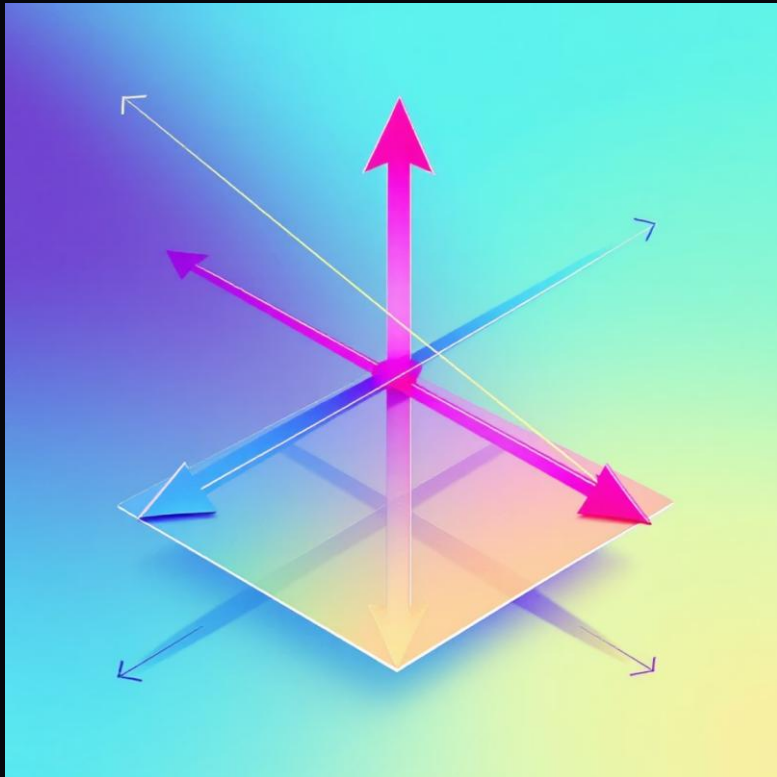
이동 (Translation)

객체의 위치를 변경합니다. 동차 좌표계를 사용하여 행렬 곱셈으로 표현할 수 있습니다

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

외적(Cross Product) — 3D 공간에서의 수직 벡터

두 벡터에 모두 수직인 새로운 벡터를 찾는 연산. 3D 공간에서 방향과 회전을 제어하는 핵심 도구입니다.



오른손 법칙으로 외적 벡터의 방향을 쉽게 알 수 있습니다.

첫 번째 벡터 방향으로 오른손을 편 다음, 두 번째 벡터 방향으로 손가락을 감아쥐면 엄지손가락이 가리키는 방향이 외적 벡터의 방향입니다.

게임 개발에서의 활용

▣ 표면 법선 계산

메시의 두 변 벡터 외적을 통해 표면의 방향(법선 벡터)을 구합니다.

▣ 회전축 결정

두 벡터 사이를 회전할 때 회전축이 될 벡터를 쉽게 찾을 수 있습니다.

▣ 방향 판정

두 벡터가 이루는 평면에 대해 특정 객체가 어느 쪽에 있는지 판단합니다.

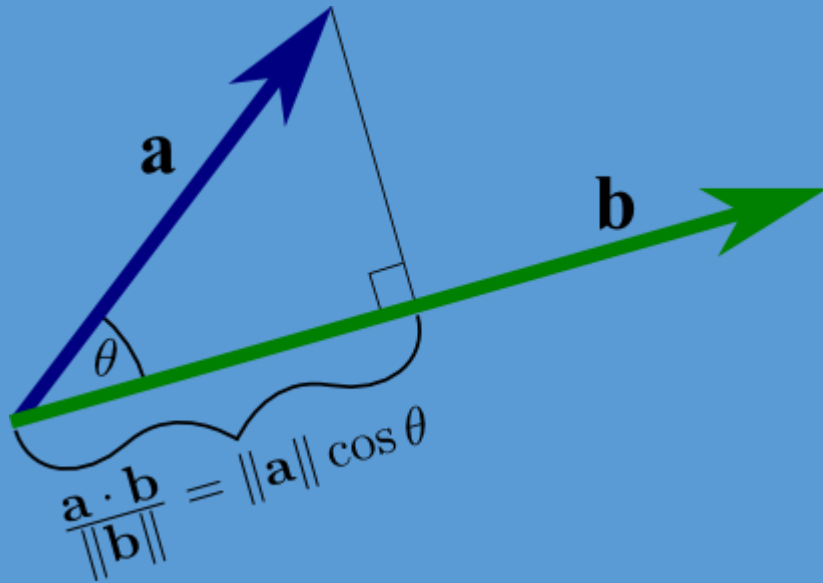
```
import numpy as np

vec_a = np.array([1, 0, 0]) # x축 방향 벡터
vec_b = np.array([0, 1, 0]) # y축 방향 벡터

cross_product = np.cross(vec_a, vec_b)
# 결과: array([0, 0, 1]) -> z축 방향 (오른손 법칙)
```


내적(Dot Product) — 두 벡터의 관계를 읽다

두 벡터가 얼마나 같은 방향을 향하는지, 그리고 어떤 관계를 가지는지 알려주는 핵심적인 연산입니다.



두 벡터의 내적은 한 벡터의 크기와 다른 벡터의 그 벡터 방향으로의 성분 크기를 곱한 값입니다. 이는 두 벡터가 얼마나 같은 방향을 향하는지 수치적으로 나타냅니다.

```
import numpy as np

v1 = np.array([3, 0])
v2 = np.array([2, 2])

dot_product = np.dot(v1, v2)
# 결과: 6 (두 벡터가 서로 '보는' 정도)
```

기하학적 의미

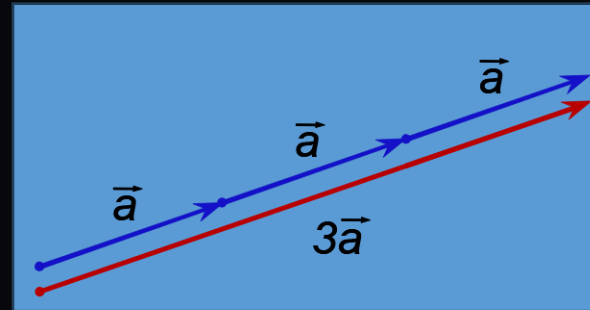
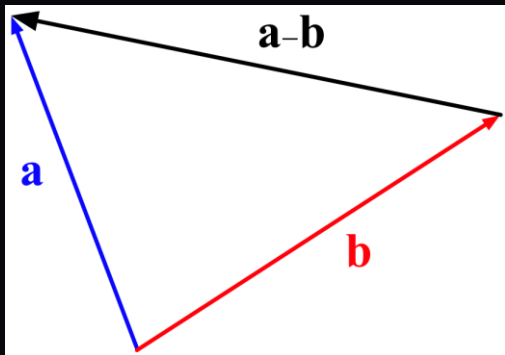
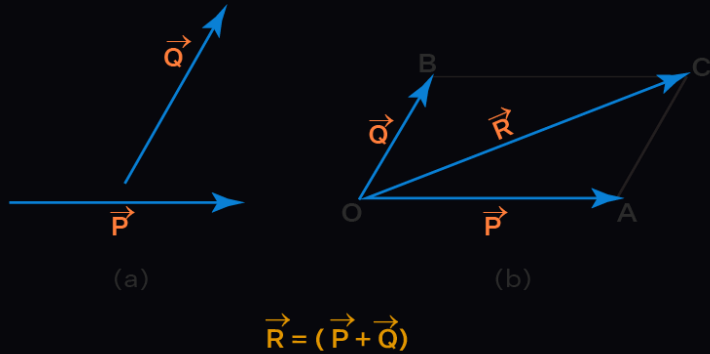
두 벡터 사이의 각도(θ)를 사용하여 $\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \|\mathbf{B}\| \cos \theta$ 로 정의되며, 두 벡터의 유사성을 측정합니다.

게임 속 활용

- 조명 계산: 표면과 빛의 각도를 통해 빛의 강도 결정
- 시야 판정: 캐릭터의 시야 내에 오브젝트가 있는지 확인
- 충돌 감지: 충돌 방향 및 반사 경로 계산

벡터의 기하학적 의미 — 시각으로 이해하기

벡터의 기본 연산인 덧셈, 뺄셈, 스칼라배가 2차원 공간에서 어떻게 시각적으로 표현되는지 살펴봅시다.



→ 벡터 덧셈 (Vector Addition)

두 벡터를 연결아 놓거나, 평행사변형 법칙을 통해 시각화합니다. 두 힘이 합쳐지는 것과 같습니다.

```
import numpy as np
v1 = np.array([2, 3])
v2 = np.array([1, -2])
v_sum = v1 + v2
# 결과: array([3, 1])
```

→ 벡터 뺄셈 (Vector Subtraction)

두 벡터의 시점을 일치시킨 후, 빼는 벡터의 종점에서 시작 벡터의 종점으로 향하는 벡터로 표현됩니다.

```
v1 = np.array([2, 3])
v2 = np.array([1, -2])
v_diff = v1 - v2
# 결과: array([1, 5])
```

→ 스칼라배 (Scalar Multiplication)

벡터의 길이를 늘리거나 줄이고, 음수를 곱하면 방향을 반대로 바꿉니다. 방향은 유지된 채 크기만 변합니다.

```
v1 = np.array([2, 3])
scalar = 2
v_scaled = v1 * scalar
# 결과: array([4, 6])
```

**자 이제 한 학기 동안
수학을 눈으로 살펴 봅시다.**